

SORBONNE UNIVERSITÉ
Faculté des Sciences et Ingénierie
Département Informatique
Laboratoire LIP6

RAPPORT DE STAGE - ÉQUIPE DELYS

Traitement distribué en temps réel : Architecture SPS et évaluation avec Apache Flink

Présenté par :
Ji Stéphane

Encadré par :
Daniel Wladdimiro
PAGLIARI Alessio

Table des matières

1.Introduction 3
1.1 Contexte général du temps réel 3
1.2 Présentation de l'équipe DELYS (LIP6) 3
1.3 Encadrement et collaboration 3
1.4 Objectifs et problématique du stage 3
2. Fondamentaux et choix technologiques 4
2.1 Concepts fondamentaux des SPS 4
2.2 Comparatif des moteurs : Limites d'Apache Storm et Samza vs Avantages d'Apache Flink ... 4
2.4 Choix du benchmark: Nexmark 5
3. Conception et Implémentation de l'architecture 6
3.1 Contexte de Nexmark 6
3.2 Architecture globale du pipeline de données 6
3.3 Implémentation des requêtes Nexmark (Opérateurs Stateless vs Stateful) 6
3.4 Stratégie de partitionnement et gestion de l'état (Checkpointing) 6
4. Expérimentation 8
4.1 Choix des métriques d'évaluation 8
4.2 Évaluation de référence : Opérateurs Stateless vs Stateful 8
4.2 Expérience importante/finale 9
Limites et Perspectives 11

1.Introduction

1.1 Contexte général du temps réel

Le traitement des flux en temps réel est devenu un domaine crucial de l'informatique moderne, particulièrement dans les contextes où la rapidité et la précision des informations sont essentielles. Face à des volumes de données en constante augmentation, les architectures traditionnelles montrent leurs limites. L'utilisation de systèmes de traitement de flux, ou Stream Processing Systems (SPS), permet de gérer et d'analyser des flux continus pour offrir des réponses immédiates aux événements d'un système.

1.2 Présentation de l'équipe DELYS (LIP6)

C'est dans ce contexte technologique stimulant que j'ai effectué mon stage au sein du Laboratoire d'Informatique de Paris 6 (LIP6). Plus précisément, j'ai intégré le département « Architecture, systèmes et réseaux », dont les travaux portent sur les infrastructures d'exécution et de communication complexes. Ce département étudie des architectures hétérogènes et distribuées, devant répondre à des problématiques avancées telles que la dynamicité, la consommation d'énergie ou les contraintes de temps réel.

1.3 Encadrement et collaboration

J'ai réalisé ce projet sous la supervision de mes tuteurs, Alessio Pagliari et Daniel Wladdimiro. Mon travail s'est également inscrit au sein d'une équipe regroupant plusieurs stagiaires travaillant dans la même salle. En particulier, j'ai collaboré avec Davy Tsy et Biancca, deux étudiants du master résaux de Sorbonne Université. Cette collaboration était indispensable car nos missions respectives formaient un écosystème distribué complet : pendant que j'étais chargé de la conception du moteur SPS, mes collègues développaient le générateur d'ingestion de données en amont et préparaient l'infrastructure Cloud (DevOps) nécessaire au déploiement de l'ensemble de notre architecture.

1.4 Objectifs et problématique du stage

L'objectif principal de mon stage était de concevoir et de développer une application SPS capable de traiter en temps réel les données provenant d'un système de file d'attente. Le projet a débuté par l'étude des concepts fondamentaux liés aux graphes acycliques dirigés (DAG) et l'apprentissage d'API spécifiques de frameworks reconnus tels qu'Apache Storm, Apache Flink ou Apache Samza. La mission exigeait de développer au moins une application avec état (stateful) et une application sans état (stateless), puis de les évaluer, notamment en répondant aux requêtes du benchmark académique Nexmark.

2. Fondamentaux et choix technologiques

2.1 Concepts fondamentaux des SPS

Le traitement de flux en temps réel exige des architectures capables d'ingérer et d'analyser des données à la volée. Pour répondre à ce besoin, les applications Stream Processing Systems (SPS) s'appuient sur des concepts théoriques et architecturaux fondamentaux.

- **Le modèle DAG (Graphe Acyclique Dirigé)**: La structure d'une application SPS est fondamentalement modélisée sous la forme d'un DAG. Dans ce graphe, les sommets représentent les opérateurs de calcul (filtrage, agrégation, jointure) et les arêtes définissent la direction du flux de données continu.
- **Traitement avec et sans état (Stateful/Stateless)**: Une infrastructure complète nécessite de gérer différents types de traitements. Les opérateurs stateless traitent chaque événement indépendamment, tandis que les opérateurs stateful doivent conserver un contexte en mémoire pour calculer des métriques sur une fenêtre temporelle ou un historique.
- **Backpressure**: Dans un système distribué, il arrive que les sources émettent des données plus rapidement que la capacité de traitement des opérateurs avals. Le mécanisme de backpressure permet de propager un signal de ralentissement vers l'amont pour éviter la saturation de la mémoire et la perte de paquets.
- **Sémantique de livraison**: En cas de panne d'un composant requis pour le déploiement de l'application distribuée, le système doit garantir qu'aucun événement n'est perdu ou compté en double. La sémantique Exactly-once (exactement-une-fois) est le standard académique actuel, nécessitant des algorithmes de checkpointing distribués.

2.2 Comparatif des moteurs : Limites d'Apache Storm et Samza vs Avantages d'Apache Flink

La littérature propose plusieurs frameworks SPS majeurs pour orchestrer ces graphes de calcul, notamment Apache Storm, Apache Flinket APache Samza. Il est impératif de comprendre les API spécifiques de ces framesworks ainsi que leurs composants de déploiement pour choisir la solution la plus adaptée.

- **Apache Storm**: Storm offre une lantence très faible car il traite les événements un par un. Toutefois, son modèle de base ne gère pas nativement l'état de manière robuste et ne propose par défaut qu'une sémantique At-least-once. Sa surcouche pour obtenir l'exactement-une-fois dégrade fortement le débit (via un traitement par micro-lots).
- **Apache Samza**: Samza est un excellent framework fortement couplé à APache Kafka. Son approche de gestion d'état locale est performante, mais son couplage restrictif et son modèle de déploiement (souvent lié à Hadoop YARN) le rendent moins flexible pour noter architecture cloud hétérogène.
- **Apache Flink**: Flink représente l'état de l'art actuel dans la recherche et l'industrie.
  - Architecture native: Contrairement aux systèmes par micro-lots, Flink est un véritable moteur de flux continu.
  - Gestion de l'état et Tolérance aux pannes: Il intègre un mécanisme de checkpointing asynchrone (basé sur l'algorithme de Chandy-Lamport) offrant une sémantique Exactly-once sans sacrifier les performances.
  - Adéquation au projet: Flink permet de développer facilement des applications avec et sans état tout en s'intégrant parfaitement avec un système de file d'attente (comme Kafka).

En conclusion, après avoir étudié le fonctionnement des différents cadres SPS, Apache Flink s'impose comme le meilleur choix technologique pour garantir à la fois un haut débit, une faible latence et une fiabilité à l'échelle.

2.4 Choix du benchmark: Nexmark

L'évaluation d'un système distribué nécessite des métriques standardisées et comparables. Pour mesurer les performances de notre architecture Flink, nous avons retenu le **Nexmark Benchmark**.

Nexmark est un standart académique qui simule un système d'enchères en ligne en temps réel (générant des flux continus de personnes, d'enchères et d'objets). Il permet de tester exhaustivement les capacités du moteur SPS à travers une série de requêtes de complexité variable.

Durant le stage, plusieurs applications ont été développées pour répondre spécifiquement aux requêtes de ce benchmark, divisées en deux grandes catégories:

- **Opérations sans état (Stateless)**: Ces requêtes testent le débit brut du système sur des transformations simples.
- **Opérations avec état (Stateful)**: Ces requêtes sont cruciales pour évaluer la gestion de la mémoire, le fenêtrage temporel et le checkpointing.

L'implémentation de ces requêtes Nexmark permet de soumettre notre application SPS à un stress mécanique réaliste, préparant ainsi le système à des cas d'étude appliqués,

3. Conception et Implémentation de l'architecture

3.1 Contexte de Nexmark

Le simulateur possède 3 source de type de données : Bid, Person et Auction

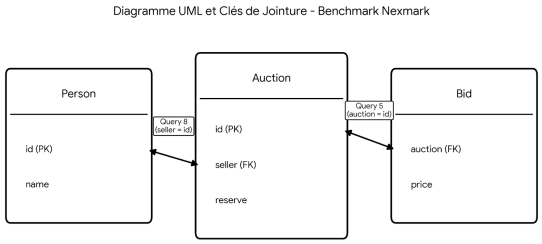


Fig. 1. – Source de donnée de Nexmark

Nexmark propose 8 requêtes à réaliser.

3.2 Architecture globale du pipeline de données

L'architecture de notre système de traitement de flux (SPS) repose sur un pipeline classique en trois phases (Ingestion, Traitement, Restitution) :

- Ingestion: Les données sont générées séquentiellement par les fichiers BidSource, PersonSource et AuctionSource.
- Traitement: Apache Flink consomme ces flux en temps réel. La logique métier est modélisée sous la forme d'un Graphe Acyclique Dirigé (DAG), où chaque nœud représente un opérateur de calcul (filtre, fenêtrage, agrégation).
- Restitution: Une fois traités, les événements finaux et les métriques de performance sont dirigés les résultats sont exportés sous forme de fichiers CSV.

3.3 Implémentation des requêtes Nexmark (Opérateurs Stateless vs Stateful)

Les requêtes ont été implémentées à l'aide de l'API DataStream de Flink pour opposer deux paradigmes de calcul :

- **Query 2 (Stateless)**: Un filtrage simple (via l'opérateur .filter()). Chaque événement est évalué isolément à la volée. L'absence de mémorisation permet d'atteindre un débit maximal avec une latence quasi-nulle.
- **Query 5 (Stateful)**: L'identification des enchères les plus populaires via une agrégation par fenêtre. Elle nécessite de maintenir un état en mémoire pour compter les événements. Pour garantir la stabilité sur l'environnement de test local et éviter la surcharge processeur, l'implémentation initiale utilisant des fenêtres glissantes (Sliding Windows) a été simplifiée au profit de fenêtres fixes (Tumbling Windows).

3.4 Stratégie de partitionnement et gestion de l'état (Checkpointing)

- Partitionnement : La distribution de la charge sur les différents cœurs est assurée par l'opérateur .keyBy(). Les flux sont segmentés logiquement selon une clé spécifique (comme l'ID de l'enchère), garantissant que tous les événements liés à cette clé sont routés et traités par la même sous-tâche.
- Gestion de l'état et Checkpointing : Les données accumulées dans les fenêtres sont gérées par les State Backends de Flink. Pour assurer la tolérance aux pannes (sémantique Exactly-once), Flink

réalise des checkpoints (snapshots asynchrones de l'état global). Lors de nos tests locaux, ces sauvegardes ont été désactivées afin d'économiser la mémoire Heap et de prévenir les arrêts prématurés liés aux contraintes matérielles.

4. Expérimentation

4.1 Choix des métriques d'évaluation

Afin de comparer objectivement les architectures avec et sans état, notre évaluation s'appuie sur des métriques ciblant spécifiquement le surcoût lié à la gestion de la mémoire et de la fiabilité.

- Débit maximal soutenable : Cette mesure évalue l'impact direct des accès à l'état sur la capacité de traitement avant saturation. L'écart de débit entre les deux systèmes permet de quantifier le coût réel des opérations de lecture/écriture de l'état
- Latence au 99e centile (p99) : Contrairement à la moyenne qui lisse les résultats, cette métrique est indispensable pour mettre en évidence les micro-ralentissements et pics de latence liés à la sérialisation de l'état
- Mémoire : La surconsommation de ressources pour une logique métier identique.

L'expérience en local a été faite depuis mon pc qui possède les caractéristiques :
- 16 Go RAM
- AMD Ryzen™ 7 5700U with Radeon™ Graphics × 16

n

4.2 Évaluation de référence : Opérateurs Stateless vs Stateful

Afin d'isoler le surcoût inhérent à la gestion de l'état, nous soumettons un flux d'événements strictement identique (Bid) à deux logiques de traitement distinctes, toutes deux utilisant le même partitionnement de clés (keyBy) :

- Traitement sans état: Un filtrage statique appliquant un prédicat d'exclusion sans conservation d'historique.
- Traitement avec état: Un filtrage dynamique comparant l'enchère courante à la valeur maximale historique de l'objet, impliquant la lecture et la mise à jour systématiques d'un état local en mémoire.

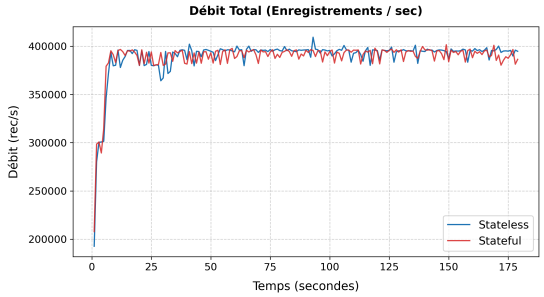


Fig. 2. – Graphe de débit

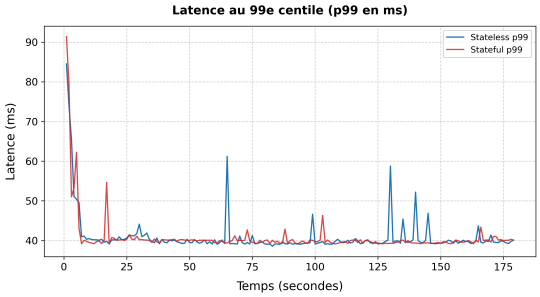


Fig. 3. – Graphe de latence P99

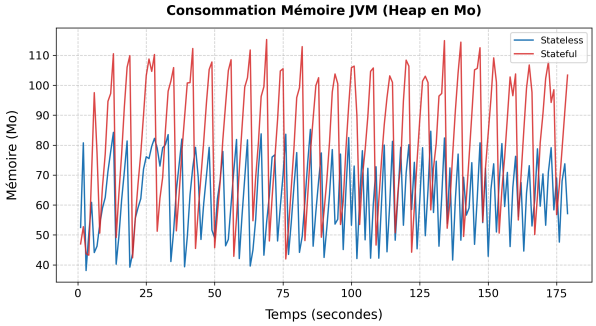


Fig. 4. – Graphe de mémoire heap

L'analyse des métriques d'exécution met en avant les architecturales suivantes :

- Débit brut équivalent : Les deux systèmes atteignent un plafond de traitement quasi identique, avec une moyenne d'environ 389 000 enregistrements/seconde pour le stateless et 388 000 pour le stateful. Cela démontre qu'à faible cardinalité, l'accès au ValueState ne crée pas de goulot d'étranglement CPU.
- Latence de traitement: La latence au 99e centile (p99) montre une phase de « chauffe » de la JVM (JIT) lors des 10 premières secondes (jusqu'à 90 ms) avant de se stabiliser autour de 40 ms pour les deux modèles. L'impact de la sérialisation de l'état sur la latence reste marginal dans ce scénario.
- Empreinte mémoire et Garbage Collector: La différence structurelle majeure se situe sur la mémoire JVM (heap_moyen_mo). L'application stateful consomme en moyenne 30 % de mémoire en plus (82 Mo contre 63 Mo). Surtout, la courbe du stateful présente des cycles en « dents de scie » de forte amplitude (de 40 à 115 Mo), illustrant l'accumulation continue des objets d'état en mémoire suivie de cycles lourds de nettoyage par le Garbage Collector, contrairement au profil plus contenu du stateless.

4.2 Expérience importante/finale

Afin d'isoler et de quantifier le coût intrinsèque de la gestion d'état dans Apache Flink, notre campagne d'évaluation finale met en regard deux requêtes représentatives du benchmark Nexmark aux comportements délibérément opposés :

- **Requête 2 (SELECTION)** - Traitement sans état (stateless) : elle applique un simple filtrage suivi d'une projection sur les identifiants d'enchères cibles. Traitant chaque événement indépendamment au fil de l'eau, elle sert de ligne de base (baseline) pour mesurer l'empreinte minimale du moteur sans rétention de données.
- **Requête 5 (HOT_ITEMS)** - Traitement à état lourd (stateful) : elle identifie périodiquement les enchères enregistrant le plus grand volume d'offres via des fenêtres temporelles glissantes combinées à un opérateur de tri. Elle sollicite intensivement le partitionnement par clé, les mécanismes de synchronisation temporelle (watermarks, timers) et le State Backend.

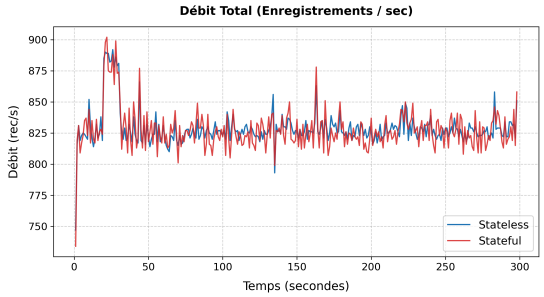


Fig. 5. – Graphe de débit

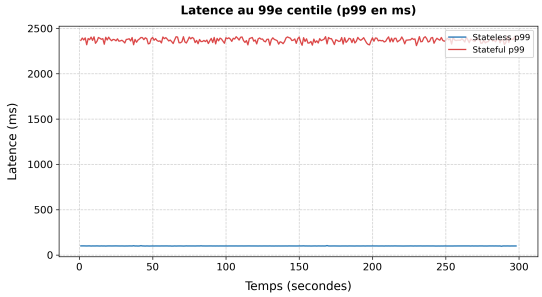


Fig. 6. – Graphe de latence P99

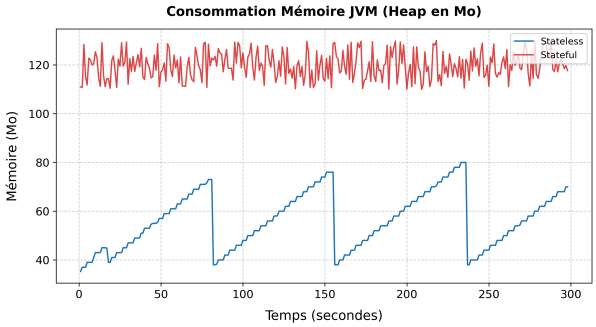


Fig. 7. – Graphe de mémoire heap

Gestion de l'état et empreinte mémoire JVM : Q2 se limite à un pipeline sans état (filter puis map), ce qui engendre uniquement des allocations éphémères rapidement collectées par le GC sous forme de dents de scie (38–80 Mo). À l'inverse, Q5 emploie un parallélisme de 4, des fenêtres glissantes partitionnées par clé et un opérateur à état explicite (ListState dans TopAuctionProcessFunction) ; ce maintien continu des fenêtres et des décomptes dans le State Backend sature le tas JVM avec un plancher permanent de 110 à 130 Mo.

Latence: Q2 traite chaque tuple au fil de l'eau dès son ingestion avec une latence quasi instantanée (100 ms). Q5 impose en revanche une attente structurelle de synchronisation : elle introduit un désordonnancement toléré de 2 secondes sur les watermarks, une durée de fenêtre de 15 secondes et un déclenchement différé via des timers temporels, ce qui décale mathématiquement l'émission des résultats et bloque la latence p99 autour de 2 350 ms.

Débit borné par la source d'ingestion : Malgré la complexité algorithmique accrue de Q5 (agrégation glissante, double partitionnement keyBy et tri du maximum), les deux topologies affichent un débit identique d'environ 825 rec/s. Cette parité s'explique par le fait que les métriques d'entrée sont relevées immédiatement après la source synthétique (BidSource), révélant que le débit est intégralement bridé par le générateur d'événements en amont et ne subit pas encore de rétro-pression (backpressure) bloquante en aval.

Limites et Perspectives

L'objectif initial de cette étude visait une évaluation comparative exhaustive incluant la Query 8 (jointure de flux avec état), la Query 5 (agrégation fenêtrée) et la Query 2 (filtrage sans état) du benchmark Nexmark. Néanmoins, la comparaison complète n'a pas pu être menée à son terme en raison de plusieurs défis techniques et temporels :

Complexité de la ligne de référence (Baseline) : Établir une base de comparaison équitable entre des requêtes aux sémantiques si différentes s'est avéré particulièrement complexe. La Query 8 implique une jointure temporelle entre deux flux distincts (enchères et personnes). Ce type d'opérateur exige le maintien d'un état mémoire nettement plus massif qu'une simple fenêtre d'agrégation (Query 5). Aligner les débits d'ingestion pour comparer rigoureusement leurs latences respectives provoquait une asymétrie de charge que notre protocole de test initial ne parvenait pas à lisser.

Goulot d'étranglement matériel : Sur notre environnement local restreint, l'exécution de la Query 8 entraînait rapidement une saturation de la mémoire (Heap JVM) et des délais d'attente critiques (timeouts Pekko), faussant ainsi les mesures de débit et de latence face à la Query 2.

Contraintes temporelles : L'optimisation requise pour stabiliser une jointure de flux (ajustement fin du State Backend, tuning des filigranes temporels ou watermarks) exigeait un investissement en temps dépassant le cadre imparti à ce stage.

Perspectives : Pour pallier ces limites, les travaux futurs devront s'orienter vers le déploiement de cette architecture sur un véritable cluster distribué (par exemple via Kubernetes/GCP, tel que modélisé dans notre architecture globale). Un tel environnement permettra d'isoler les ressources, de stabiliser l'exécution de la Query 8, et d'étudier plus en profondeur l'impact des algorithmes de tolérance aux pannes (checkpointing asynchrone) sur la latence de bout en bout.